

Step 0: Familiarity with organizational tools for scientific computing

Zan Ahmad

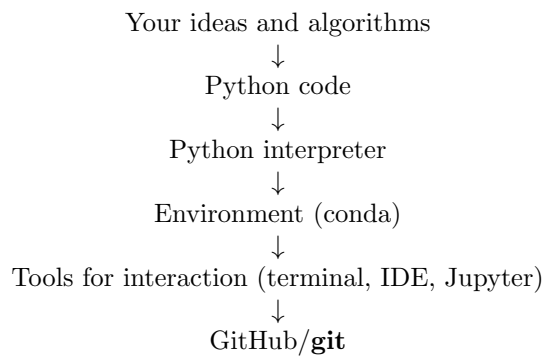
January 2026

1 Python Tooling: How Everything Fits Together

Learning Python involves more than learning a programming language. It also requires understanding the tools used to write, run, and manage code. This section explains the roles of the command line, conda, Jupyter notebooks, IDEs, and Python distributions, and clarifies when each should be used.

1.1 The Big Picture

Scientific programming can be understood as a layered system:



Each tool plays a distinct role. None of them replace the others.

1.2 The Command Line (Terminal)

The command line is a text-based interface for interacting directly with the operating system.

Common commands include:

```
python my_script.py
conda activate sci-python
jupyter lab
```

The command line is used to:

- Run Python programs
- Activate environments
- Install packages
- Launch tools such as Jupyter or VS Code

The command line is *not* Python itself, and it is not where code is typically written. Instead, it is how tools are launched and controlled.

Key idea: The command line is how you *start* and *manage* programming tools, not where you think through problems.

1.3 Conda and Environments

Conda is an environment and package manager. It is not Python itself.

A *Python environment* is a self-contained collection of:

- A specific Python interpreter
- A specific set of installed libraries
- Specific library versions

Different projects often require different environments. Without environments, package conflicts and reproducibility problems arise.

Conda is used to:

- Create environments
- Activate environments
- Install and update packages

Example:

```
conda create -n sci-python python=3.12
conda activate sci-python
conda install numpy scipy matplotlib
```

Key idea: Conda determines *which Python interpreter* and *which libraries* are used when code is executed.

1.4 The Python Interpreter

The Python interpreter is the program that executes Python code. When running:

```
python my_script.py
```

you are instructing the operating system to use the Python interpreter associated with the currently active environment.

Each conda environment has its own interpreter. If the wrong environment is active, imports may fail or unexpected behavior may occur.

Key idea: If something does not work, the first thing to check is which environment and interpreter are active.

1.5 Jupyter Notebooks

A Jupyter notebook is an interactive document that executes Python code in cells. It allows code, text, equations, and figures to coexist.

Jupyter notebooks are particularly useful for:

- Learning and experimentation
- Data exploration
- Visualization
- Prototyping ideas
- Teaching and demonstrations

Jupyter notebooks are less suitable for:

- Large software projects
- Long-term reproducible pipelines
- Version-controlled production code

Key idea: Jupyter is a space for thinking and exploration, not a final product.

1.6 IDEs (Integrated Development Environments)

An IDE is a full workspace for writing and maintaining code. Common examples include VS Code, Spyder, and PyCharm.

An IDE typically provides:

- A code editor
- Syntax highlighting and auto-completion

- Debugging tools
- File navigation
- Integrated terminal access

IDEs are best used for:

- Writing reusable `.py` files
- Organizing projects
- Debugging non-trivial logic
- Maintaining code over time

1.7 GitHub and Version Control

GitHub is a platform for hosting and collaborating on code using a tool called *git*. Git is a version control system that records changes to files over time, allowing you to track history, revert mistakes, and collaborate safely.

It is important to distinguish between the two:

- **git** is the version control tool
- **GitHub** is a website that hosts git repositories

1.8 What Problem Does GitHub Solve?

When working on code, especially over time or with others, common problems arise:

- Accidentally deleting or breaking working code
- Losing track of which version is correct
- Needing to collaborate without overwriting each other's work

GitHub solves these problems by keeping a complete history of your project and allowing changes to be saved incrementally.

1.9 Repositories

A *repository* (or *repo*) is a folder that contains:

- Your code files
- A record of all past changes
- Metadata describing the project

Repositories can be:

- Local (on your computer)
- Remote (hosted on GitHub)

1.10 Basic Git Workflow

A typical workflow using git and GitHub looks like:

1. Edit files locally (in an IDE or editor)
2. Save a snapshot of changes (`commit`)
3. Upload changes to GitHub (`push`)

This allows you to experiment freely while maintaining a recoverable history.

1.11 Common Beginner Misconceptions About GitHub

- “GitHub runs my code” — GitHub stores code; Python runs it.
- “GitHub replaces my computer” — your code still lives locally.
- “I need GitHub for everything” — use it when work matters.

Key idea: GitHub is a safety net and collaboration tool, not a programming environment.

1.12 How These Tools Work Together

A typical workflow looks like:

1. Open a terminal
2. Activate a conda environment
3. Launch Jupyter or an IDE
4. Pull from Github to ensure you’re working on the most up to date version of the project
5. Write and run Python code
6. Iterate and debug
7. Push to Github at the end of a session to save your work.

1.13 When to Use What

Task	Tool
Install packages	Conda + command line
Explore ideas or learn	Jupyter notebook
Write structured programs	IDE + <code>.py</code> files
Run code	Command line or IDE
Fix import errors	Check conda environment

1.14 Common Misconceptions

- “Jupyter is Python” — Jupyter runs Python.
- “Conda is an IDE” — Conda manages environments.
- “VS Code replaces the terminal” — VS Code uses the terminal.
- “Environments are optional” — environments are essential.

1.15 Anaconda vs. Miniconda

Anaconda and Miniconda are both Python distributions that include conda. They differ only in what comes preinstalled.

Anaconda Anaconda includes Python, conda, and a large collection of scientific packages preinstalled.

- Large installation size
- Minimal setup effort
- Convenient for beginners and teaching

Anaconda can be thought of as a fully furnished apartment.

Miniconda Miniconda includes Python and conda, but very few additional packages.

- Lightweight installation
- Explicit control over dependencies
- Preferred for research and reproducibility

Miniconda can be thought of as an empty apartment.

Overall: Anaconda prioritizes convenience. Miniconda prioritizes precision and control.